

Attack Surface for a Quantization-Conditioned RAG Index Backdoor: A Code-Level Audit of FAISS, LangChain, LlamaIndex, Milvus, Qdrant, and Weaviate

Fall 2026 Conversion-Integrity Project

27 September 2026

Snapshot audited. All paths are relative to `RAG-Related/`. Commits: faiss `fdb9535c1`, langchain `d7508191bc`, llama_index `169e450aa`, milvus `acd8a6770e`, qdrant `6ab21cac1`, weaviate `e3f4df7ba8`. Line numbers refer to these commits. “Not found” means we searched the repository and found no such code. It does not cover code outside the repository.

1 Verdict

The four-part surface (Q1–Q4) exists, but not in the form “RAG stacks ship PQ by default”. That claim is **false** for five of the six repositories. **Q1:** PQ/SQ codebook construction exists in FAISS (C++ k-means per sub-vector), Qdrant (own Rust PQ/SQ), and Weaviate (own Go PQ/SQ/RQ). Milvus exposes PQ/SQ index types but delegates the implementation to the external `knowhere` library, which is not in the repo. LangChain and LlamaIndex implement no quantization and delegate to a backend. **Q2:** PQ is opt-in everywhere. Scalar quantization is on by default in exactly one place: **Milvus**. There, any vector index created without an explicit `index_type`, including the SDK’s quick-setup path, resolves to `HNSW_SQ` with `sq_type=SQ4U` and an FP16 refine stage. Weaviate and Qdrant leave compression off unless the collection or server is configured for it. Once PQ is enabled in Weaviate, it trains automatically when a shard passes `trainingLimit=100,000` vectors (a scale trigger). **Q3:** every system has a shippable compressed artifact. The artifacts are: a FAISS `write_index` file (codebooks and codes, with no raw vectors for IVFPQ); a LlamaIndex persist directory (which removes embeddings from the docstore, so the FAISS file is the only copy of the vectors); a Qdrant snapshot tarball (`quantized.meta.json` holds the centroids as plain JSON); Weaviate backups (HNSW commit logs carry the PQ encoders); and Milvus snapshots, including `RestoreExternalSnapshot`, which copies prebuilt index files. **Q4:** no repository checks a loaded quantized index against the source embeddings. The only checks we found are structural (shape and size), identity-based (Milvus build-ID ownership), or optional whole-file SHA-256 values supplied by the same party that ships the file (Qdrant). The real limit on the attack is *rescoring*. Weaviate re-ranks compressed candidates with full-precision vectors by default, and Milvus’s default index has an FP16 refine stage. Both reduce the attack to candidate-set suppression. Qdrant’s PQ/SQ do *not* rescore by default, and FAISS IVFPQ never does unless the index is wrapped in `IndexRefine`. **Strongest:** FAISS IVFPQ loaded through LlamaIndex’s `FaissVectorStore.from_persist_dir` (Section 5). Next come Qdrant with PQ enabled (no default rescoring, snapshot recovery from a URL) and Milvus’s default `HNSW_SQ` (quantized by default, but with refine). **Weakest:** LangChain core, whose in-repo stores are exact and whose FAISS wrapper is not in this repo, and Weaviate, which rescoring by default. The honest threat model is therefore an *explicitly configured or scale-triggered* deployment, with Milvus SQ as the one default-on case, rather than a default-on PQ supply chain.

2 Summary table

Repo	Q1 PQ/SQ exists	Q2 default vs. opt-in (exact default)	Q3 shippable compressed artifact	Q4 verification on load	Q5 seed / determinism	Q6 direct codebook override
FAISS	yes. faiss/faiss/impl/ProductQuantizer.cpp:130-206 (k-means per sub-vector); faiss/faiss/IndexIVFPQ.cpp:76-95; OPQ faiss/faiss/VectorTransform.cpp:1246.	opt-in. There is no default index: <code>index_factory</code> needs a description (faiss/faiss/index_factory.h:17-20); "Flat" gives exact IndexFlat (faiss/faiss/index_factory.cpp:809-811).	yes. <code>write_index</code> stores centroids and codes (faiss/faiss/impl/index_write.cpp:185-190, 801-810).	no. Shape checks only (faiss/faiss/impl/index_read.cpp:735-768).	seeded: <code>seed=1234</code> (faiss/faiss/Clustering.h:47-50); OPQ 1234 (faiss/faiss/VectorTransform.cpp:1262). Seed and cp are <i>not serialized</i> .	yes. <code>set_params</code> (faiss/faiss/impl/ProductQuantizer.cpp:76-80), <code>Train_hot_start</code> , <code>add_sa_codes</code> (faiss/faiss/Index.h:430), <code>update_entries</code> .
LangChain	no in repo. FAISS is a lazy import from <code>langchain_community</code> , which is not vendored (langchain/libs/langchain/langchain_classic/vectorstores/faiss.py:11).	no quantization. <code>InMemoryVectorStore</code> is exact cosine (langchain/libs/core/langchain_core/vectorstores/in_memory.py:314); the Qdrant partner creates plain <code>VectorParams</code> (langchain/libs/partners/qdrant/langchain_qdrant/qdrant.py:944-982).	partial. <code>InMemoryVectorStore</code> : <code>dump/load</code> is full-precision JSON (langchain/libs/core/langchain_core/vectorstores/in_memory.py:517-546); <code>from_existing_collection</code> attaches to a pre-built Qdrant collection.	no. The Qdrant partner validates only size and distance (langchain/libs/partners/qdrant/langchain_qdrant/qdrant.py:1230, 1240).	n/a (no training code).	n/a in repo.
LlamaIndex	no own PQ. It wraps any <code>faiss.Index</code> (llama_index/llama-index-integrations/vector_stores/llama-index-vector-stores-faiss/llama_index/vector_stores/faiss/base.py:65-82).	no quantization by default: <code>SimpleVectorStore</code> (exact) (llama_index/llama-index-core/llama_index/core/storage/storage_context.py:105-107). The Milvus integration forces FLAT (.../milvus/base.py:1406).	yes. <code>persist</code> calls <code>faiss.write_index</code> (.../faiss/base.py:171), and the docstore <i>removes</i> embeddings (llama_index/llama-index-core/llama_index/core/indices/vector_store/base.py:196-203).	no. <code>from_persist_path</code> calls <code>faiss.read_index</code> and trusts the result (.../faiss/base.py:100-117).	Inherits FAISS.	yes , inherited: any user-supplied FAISS object is accepted.
Milvus	partial. Types <code>IVF_PQ</code> , <code>HNSW_PQ</code> , <code>HNSW_SQ</code> , <code>SCANN</code> (milvus/client/index/common.go:46-57); the implementation is in external <i>knowhere</i> (milvus/internal/core/thirdparty/knowhere/CMakeLists.txt:17-18).	SQ on by default. <code>AUTOINDEX</code> resolves to <code>HNSW_SQ/SQ4U/refine FP16</code> (milvus/pkg/util/paramtable/autoindex_param_nocuda.go:26) when no <code>index_type</code> is given (milvus/internal/proxy/task_index.go:379-386). PQ is opt-in.	yes. Snapshots copy index files (milvus/internal/datacoord/copy_segment_task.go:803-806, 990-1014); <code>RestoreExternalSnapshot</code> (milvus/internal/proxy/snapshot_impl.go:213).	no content check. Build-ID <i>ownership</i> only (milvus/internal/datacoord/copy_segment_task.go:1316-1320).	Not in repo (nowhere).	no public API (no centroid or codebook symbol in <code>milvus/internal/proxy</code>). Only by crafting index files.

Repo	Q1 PQ/SQ exists	Q2 default vs. opt-in (exact default)	Q3 shippable compressed artifact	Q4 verification on load	Q5 seed / determinism	Q6 direct codebook override
Qdrant	yes. PQ (qdrant/lib/quantization/src/encoded_vectors_pq.rs:342-400), k-means (qdrant/lib/quantization/src/kmeans.rs:9-27), SQ, BQ, TQ.	opt-in: quantization: null (qdrant/config/config.yaml:250-252). HNSW is uncompressed. Quantization is built only for indexed segments (threshold qdrant/lib/shard/src/optimizers/config.rs:17).	yes. The snapshot tar includes the quantized files (qdrant/lib/segment/src/segment/snapshot.rs:304-312); recover from a URL (qdrant/src/actix/api/snapshot_api.rs:251).	no. The files are loaded as-is (qdrant/lib/segment/src/vector_storage/quantized/quantized_vectors/load.rs:32-35); an optional user-supplied SHA-256 covers the whole file. PQ/SQ do not rescore by default (qdrant/lib/segment/src/vector_storage/quantized/quantized_vectors/accessors.rs:16-22).	unseeded: rand::rng() for sampling (qdrant/lib/quantization/src/encoded_vectors_pq.rs:365-366). Nothing recorded.	no API. quantized.meta.json is plain JSON (qdrant/lib/quantization/src/encoded_vectors_pq.rs:47-51), so it can be edited in the snapshot.
Weaviate	yes. PQ (weaviate/adapters/repos/db/vector/compressionhelpers/product_quantization.go:401-457), k-means encoder, SQ, RQ, BQ.	opt-in. Default index is HNSW (weaviate/entities/vectorindex/config.go:25); PQ, SQ, RQ, and BQ are all false (weaviate/entities/vectorindex/hnsw/pq_config.go:28); DEFAULT_QUANTIZATION defaults to empty (weaviate/usecases/config/environment.go:633-638). Once enabled, PQ trains automatically at 100k (weaviate/adapters/repos/db/vector_index_queue.go:285-302).	yes. The backup includes HNSW commit logs (weaviate/adapters/repos/db/vector/hnsw/backup.go:77-79), which carry the PQ encoders (weaviate/adapters/repos/db/vector/hnsw/commitlog/logger.go:84-103).	no check, but rescores by default with full vectors (weaviate/adapters/repos/db/vector/hnsw/search.go:188-195, 822-852).	random: Seed: rand.Uint64() (weaviate/adapters/repos/db/vector/kmeans/kmeans.go:163). Not persisted (weaviate/entities/vectorindex/compression/pq_data.go:41-50).	partial. Internal Go constructors take centers (.../compressionhelpers/kmeans_encoder.go:41-45). There is no REST path, so the attacker has to craft commit logs.

3 Per-repository evidence

3.1 FAISS

Q1 (PQ construction). `faiss/faiss/impl/ProductQuantizer.h:24-29` documents the construction: “*PQ is trained using k-means, minimizing the L2 distance to centroids.*” The centroid table is a public `std::vector<float>` centroids with layout (M, k_{sub}, d_{sub}) (`faiss/faiss/impl/ProductQuantizer.h:54-56`). Training slices each sub-vector and runs an independent Clustering for each m :

```
// faiss/faiss/impl/ProductQuantizer.cpp:143-189
for (size_t m = 0; m < M; m++) {
    ... memcpy(xslice.get() + j * dsub, x + j * d + m * dsub, ...);
    Clustering clus(dsub, ksub, cp);
    ...
    clus.train(n, xslice.get(), assign_index ? *assign_index : index);
    set_params(clus.centroids.data(), m);
}
```

Encoding assigns each sub-vector to its nearest centroid in `compute_code` (`faiss/faiss/impl/ProductQuantizer.cpp:282`). `IndexIVFPQ` (`faiss/faiss/IndexIVFPQ.h:34`) trains a coarse quantizer and then trains the PQ on residuals: `train_encoder` calls `pq.train(n, x)` (`faiss/faiss/IndexIVFPQ.cpp:76-80`). `by_residual = true` is the default (`faiss/faiss/IndexIVFPQ.cpp:64`). The coarse k-means uses `cp.niter = 10` (`faiss/faiss/IndexIVF.cpp:45`). OPQ is `OPQMatrix` (`faiss/faiss/VectorTransform.h:258-262`), trained in `faiss/faiss/VectorTransform.cpp:1246`. SQ types are parsed from factory strings (`faiss/faiss/index_factory.cpp:162-181`).

Q2 (default). FAISS has no default index. `index_factory(d, description, metric)` requires a description (`faiss/faiss/index_factory.h:17-20`), and “Flat” maps to an exhaustive index:

```
// faiss/faiss/index_factory.cpp:808-811
// IndexFlat
if (description == "Flat") { return new IndexFlat(d, metric); }
```

`IndexFlat` “stores the full vectors and performs exhaustive search” (`faiss/faiss/IndexFlat.h:20-21`). PQ appears only when the user writes it into the description: “PQ{M}x{nbits}” (`faiss/faiss/index_factory.cpp:864-869`), “IVF{n},PQ{M}” (`faiss/faiss/index_factory.cpp:420-425`), or “OPQ{M}” (`faiss/faiss/index_factory.cpp:255-258`). **PQ is never the default.**

Q3 (artifact). The PQ is serialized together with its centroids:

```
// faiss/faiss/impl/index_write.cpp:185-190
void write_ProductQuantizer(const ProductQuantizer* pq, IOWriter* f) {
    WRITE1(pq->d); WRITE1(pq->M); WRITE1(pq->nbits);
    WRITEVECTOR(pq->centroids);
}
```

For `IndexIVFPQ`, the writer emits the `IwPQ` tag, the IVF header (which contains the coarse quantizer), the PQ, and the inverted lists of codes (`faiss/faiss/impl/index_write.cpp:801-810`). The raw float vectors are not written; only the codes are, so the full-precision embeddings are *not* in the artifact. Python exposes `serialize_index` and `deserialize_index` for byte buffers (`faiss/faiss/python/__init__.py:564,571`).

Q4 (verification). `read_ProductQuantizer` checks structure only ($M > 0$, $nbits$ in $[1, 24]$, allocation limits, and `centroids.size() == d * ksub`) (`faiss/faiss/impl/index_read.cpp:735-768`). SQ reads check the length of trained (`faiss/faiss/impl/index_read.cpp:1060-1165`). A search of `faiss/faiss/impl/index_read.cpp`, `index_write.cpp`, `io.cpp`, and `index_io.h` for checksum, CRC, hash, verify, signature, and integrity found no integrity mechanism. The only hardening is resource limits against malformed files (`faiss/faiss/index_io.h:120-137`). Exact re-ranking is available only through an explicit `IndexRefine` wrapper (`faiss/faiss/IndexRefine.h:24-39`).

Q5 (seed). `ClusteringParameters::seed = 1234`. A negative seed selects a clock-based seed (`faiss/faiss/Clustering.h:47-50`; `faiss/faiss/impl/ClusteringHelpers.cpp:28-34`). Random initialization uses `rand_perm(..., actual_seed + 1 + redo * 15486557L)` (`faiss/faiss/Clustering.cpp:192,211`). OPQ’s

initial rotation uses `float_randn(rotation, d*d, 1234)` (`faiss/faiss/VectorTransform.cpp:1262,1313`). **Consequence:** with default parameters, FAISS PQ training is reproducible given the same training set, order, and parameters. However, `cp` (seed, niter, nredo) is *not* serialized: `write_ProductQuantizer` writes only `d`, `M`, `nbits`, `centroids`, and `index_write.cpp` contains no `ClusteringParameters` or `cp`. reference. A verifier therefore has to guess the parameters and needs the exact training sample. We did not check whether results are bit-identical across thread counts or SIMD levels.

Q6 (direct override). Centroids can be written directly:

```
// faiss/faiss/impl/ProductQuantizer.cpp:76-80
void ProductQuantizer::set_params(const float* centroids_, int m) {
    memcpy(get_centroids(m, 0), centroids_, ksub * dsub * sizeof(centroids_[0]));
}
```

Other public routes: `Train_hot_start` uses existing centroids as the k-means initialization (`faiss/faiss/impl/ProductQuantizer.h:42; .cpp:175-178`); `frozen_centroids` keeps the supplied centroids fixed during training (`faiss/faiss/Clustering.h:38-40`); Python `copy_array_to_vector` writes arrays into `pq.centroids` (`faiss/faiss/python/array_conversions.py:135`), as in `faiss/tests/test_index_binary.py:47`. Codes can be written directly as well: `Index::add_sa_codes` (`faiss/faiss/Index.h:430`; IVF implementation `faiss/faiss/IndexIVF.cpp:213`), and `InvertedLists::update_entry/update_entries` (`faiss/faiss/invlists/InvertedLists.h:138-149`). **The attack does not need to go through k-means.**

3.2 LangChain

Q1. `LangChain` implements no PQ or SQ. Searching `langchain/libs` for `quantiz`, `ProductQuant`, and `ScalarQuant` outside tests finds only `Qdrant` pass-through parameters and one unrelated embedding comment. The FAISS wrapper is a lazy re-export from `langchain_community`:

```
# langchain/libs/langchain/langchain_classic/vectorstores/faiss.py:11
DEPRECATED_LOOKUP = {"FAISS": "langchain_community.vectorstores"}
```

The same pattern appears in `langchain/libs/langchain/langchain_classic/_init__.py:348-353`. **`langchain_community` is not in this repository**, so we could not audit `LangChain`'s FAISS default index or its `load_local/save_local` behavior from the cloned code.

Q2. `InMemoryVectorStore` (`langchain/libs/core/langchain_core/vectorstores/in_memory.py:34`) computes exact cosine over every stored vector (`langchain/libs/core/langchain_core/vectorstores/in_memory.py:314`). The `langchain_qdrant.QdrantVectorStore.from_texts` path defaults to `distance=COSINE` (`langchain/libs/partners/qdrant/langchain_qdrant/qdrant.py:357`) and creates the collection with only `VectorParams(size, distance)` (`langchain/libs/partners/qdrant/langchain_qdrant/qdrant.py:944-982`). The string `quantiz` does not occur anywhere in `langchain_qdrant/qdrant.py`. Only the deprecated `Qdrant` class (`langchain/libs/partners/qdrant/langchain_qdrant/vectorstores.py:59-60`) accepts `quantization_config`, and its docstring says “*if None - quantization will be disabled*” (`.../vectorstores.py:1309-1310`). The Chroma partner contains no quantization code (search of `langchain/libs/partners/chroma/langchain_chroma/` for `quantiz` and `pq`: none).

Q3. `InMemoryVectorStore.dump/load` writes and reads full-precision JSON (`langchain/libs/core/langchain_core/vectorstores/in_memory.py:517-546`), so nothing is compressed. `QdrantVectorStore.from_existing_collection` attaches to a collection that someone else built (`langchain/libs/partners/qdrant/langchain_qdrant/qdrant.py:434-470`). That collection could be quantized, but `LangChain` does not create the quantization.

Q4. When the collection already exists, `_validate_collection_config` checks only vector size (`.../qdrant.py:1230`) and distance (`.../qdrant.py:1240`). It reads no quantization configuration and compares nothing to source embeddings.

Q5/Q6. Not applicable in this repository.

3.3 LlamaIndex

Q1. LlamaIndex has no own PQ. FaissVectorStore takes a caller-constructed faiss_index (.../llama-index-vector-stores-faiss/llama_index/vector_stores/faiss/base.py:65-82; below, .../faiss/ abbreviates that integration directory).

Q2. With no vector store supplied, StorageContext.from_defaults creates SimpleVectorStore() (llama_index/llama-index-core/llama_index/core/storage/storage_context.py:105-107). It holds Dict[str, List[float]] embeddings (llama_index/llama-index-core/llama_index/core/vector_stores/simple.py:48-62) and runs exact get_top_k_embeddings (.../simple.py:303). The FAISS integration has no default index type: the user must pass one. The docstring and every doc example use faiss.IndexFlatL2(d) (.../faiss/base.py:54; llama_index/docs/examples/vector_stores/FaissIndexDemo.ipynb:84; llama_index/docs/src/content/docs/framework/community/integrations/vector_stores.md:487,495-496). A search of llama_index/docs for IndexIVFPQ and faiss.IndexPQ found no use. The Milvus integration overrides Milvus's server default and asks for FLAT: index_type = base_params.pop("index_type", "FLAT") (.../llama-index-vector-stores-milvus/llama_index/vector_stores/milvus/base.py:1406). The Qdrant integration creates VectorParams(size, COSINE) and forwards quantization_config, which defaults to None (.../llama-index-vector-stores-qdrant/llama_index/vector_stores/qdrant/base.py:190,810-815,835). The Weaviate integration's default schema sets no vector-index configuration (.../llama-index-vector-stores-weaviate/llama_index/vector_stores/weaviate/utils.py:100-108).

Q3 (the strongest artifact boundary we found). persist calls faiss.write_index(self._faiss_index, persist_path) (.../faiss/base.py:171), and from_persist_dir/from_persist_path read it back (.../faiss/base.py:85-117). The core index also removes embeddings before writing nodes to the docstore when the vector store does not store text (FAISS sets stores_text = False, .../faiss/base.py:61):

```
# llama_index/llama-index-core/llama_index/core/indices/vector_store/base.py:196-199
# NOTE: remove embedding from node to avoid duplication
node_without_embedding = node.model_copy()
node_without_embedding.embedding = None
```

The same code appears in the sync path at lines 237-240. In a persisted LlamaIndex+FAISS directory, the FAISS file is therefore the *only* copy of the vectors. With IVFPQ, no full-precision embedding remains anywhere in the artifact.

Q4. There is no verification:

```
# .../faiss/base.py:113-117
logger.info(f"Loading {__name__} from {persist_path}.")
faiss_index = faiss.read_index(persist_path)
return cls(faiss_index=faiss_index)
```

The index type is not checked. FaissVectorMapStore checks only that the loaded object is an IndexIDMap/IndexIDMap2 (.../faiss/map_store.py:89-94); it does not check what is inside.

Q5/Q6. These are inherited from FAISS. Any FAISS object, including one with hand-set centroids, is accepted.

3.4 Milvus

Q1. The Go SDK exposes quantized index types: IVF_PQ, IVF_SQ8, HNSW_SQ, HNSW_PQ, and SCANN (milvus/client/index/common.go:46-57). The implementation is in knowhere, which is fetched at build time from <https://github.com/zilliztech/knowhere.git> at version faff72c (milvus/internal/core/thirdparty/knowhere/CMakeLists.txt:17-18). Index builds go through cgo (milvus/internal/util/indexcgowrapper/index.go:109-119). **The PQ/SQ training code is not in this repository**, so we cannot cite it.

Q2 (the one default-on case). The CPU-build default for dense float vectors is:

```
// milvus/pkg/util/paramtable/autoindex_param_nocuda.go:26
DefaultValue: "{ \"M\": 18, \"efConstruction\": 240, \"index_type\": \"HNSW_SQ\", \"sq_type\": \"SQ4U\", \"refine\": true, \"refine_type\": \"FP16\", \"metric_type\": \"COSINE\" }",
```

The CUDA build uses GPU_CAGRA (milvus/pkg/util/paramtable/autoindex_param_cuda.go:26). The largeTopK variant is IVF_SQ8 (milvus/pkg/util/paramtable/autoindex_param.go:155), but it applies only when autoIndex.enable is true (milvus/internal/util/indexparamcheck/index_params_validation.go:181-187), and that defaults to false (milvus/pkg/util/paramtable/autoindex_param.go:82-84), a flag the code comments as “*enable only for cloud instance*” (milvus/internal/proxy/task_index.go:249). On open-source servers, the “behavior change after 2.2.9” branch applies the AUTOINDEX configuration whenever index_type is absent or equals AUTOINDEX (milvus/internal/proxy/task_index.go:311,379-386). The Go SDK’s quick setup, SimpleCreateCollectionOptions (milvus/client/milvusclient/collection_options.go:189-205), creates an index with an empty parameter map:

```
// milvus/client/milvusclient/collection_options.go:176-179
if opt.isFast && opt.indexOptions == nil {
    return []CreateIndexOption{ NewCreateIndexOption(opt.name, opt.vectorFieldName,
        index.NewGenericIndex("", map[string]string{})) } }
```

The client issues it (milvus/client/milvusclient/collection.go:52-54), so quick setup lands on HNSW_SQ/SQ4U. This is *scalar* quantization (4-bit uniform ranges), not a PQ codebook. PQ variants are opt-in by name. We did not verify pymilvus’s quick-setup path, which is not in the repo.

Q3. Milvus keeps raw vectors in binlogs and index files in separate objects. CreateSnapshot and RestoreSnapshot (milvus/internal/proxy/snapshot_impl.go:37,443) and RestoreExternalSnapshot (milvus/internal/proxy/snapshot_impl.go:213; datacoord milvus/internal/datacoord/ddl_callbacks_snapshot.go:132-143) restore through copy-segment jobs. The code says “*Index files (vector/scalar/text/JSON) need to be copied with segment data*” (milvus/internal/datacoord/copy_segment_task.go:805) and “*WHICH INDEX FILES TO COPY: the snapshot’s own index metadata wins*” (.../copy_segment_task.go:990). Built indexes, including quantizer parameters and refine data, therefore travel as files and are not rebuilt.

Q4. The only post-copy check is verifyCopiedManifestIndexOwnership. It “*binds each published build to its actual target index ID*” (milvus/internal/datacoord/copy_segment_task.go:1316-1320): an identity and path check, not a content check. We found no recomputation of index contents from binlogs on restore.

Q5. Not determinable from this repository (knowhere).

Q6. No public API. A search of milvus/internal/proxy/*.go (non-test) for centroid and codebook returns nothing. An attacker has to craft index files, for example inside an external snapshot.

3.5 Qdrant

Q1. The PQ encoder splits vectors into chunks, samples up to KMEANS_SAMPLE_SIZE = 10_000 vectors, and runs k-means with CENTROIDS_COUNT = 256 per chunk (qdrant/lib/quantization/src/encoded_vectors_pq.rs:28-31,342-400). k-means initializes from the first *k* sampled points (qdrant/lib/quantization/src/kmeans.rs:26-27). The centroids are stored in Metadata { centroids: Vec<Vec<f32>, vector_division, vector_parameters } (.../encoded_vectors_pq.rs:47-51). Compression ratios run from X4 to X64 (qdrant/lib/segment/src/types.rs:922-928). SQ is int8 with an optional quantile (qdrant/lib/segment/src/types.rs:939-957; qdrant/lib/quantization/src/encoded_vectors_u8.rs:193-205).

Q2. Quantization is off by default. The collection parameter is #[serde(default)] quantization_config: Option<QuantizationConfig> (qdrant/lib/collection/src/config.rs:342-344), and the server default is:

```
# qdrant/config/config.yaml:250-252
# Default quantization configuration.
quantization: null
```

When quantization is configured, it is built only for indexed/optimized segments. Appendable segments get quantization only under a feature flag (qdrant/lib/segment/src/types.rs:1133-1137; qdrant/lib/shard/src/optimizers/config.rs:44). Segments are indexed once they pass DEFAULT_INDEXING_THRESHOLD_KB = 10_000 (qdrant/lib/shard/src/optimizers/config.rs:17; qdrant/config/config.yaml:169). This is a size trigger.

Q3. Snapshots add every quantized-vector file to the tar (qdrant/lib/segment/src/segment/snapshot.rs:304-312). The file names are quantized.config.json, quantized.data, and quantized.meta.json (qdrant/lib/segment/src/vector_storage/quantized/quantized_vectors/config.rs:10-14). Snapshots can be uploaded or recovered from a location (qdrant/src/actix/api/snapshot_api.rs:193,251), including remote URLs with an optional API key (qdrant/lib/collection/src/operations/snapshot_ops.rs:82). The original vectors are also in the snapshot, because vector storage files are appended just before (.../snapshot.rs:295-302), so verification would be *possible*.

Q4. When a quantization configuration file exists, it is loaded without recomputation:

```
// qdrant/lib/segment/src/vector_storage/quantized/quantized_vectors/load.rs:32-35
let config_path = Self::get_config_path(path);
if config_path.exists() {
    let config: QuantizedVectorsConfig = read_json(&config_path)?;
    return Ok(Some(Self::load_impl(config, vector_storage, path)?));
}
```

EncodedVectorsPQ::load validates only the storage vector size against the metadata (qdrant/lib/quantization/src/encoded_vectors_pq.rs:144-162). The snapshot checksum is “*Optional SHA256 checksum to verify snapshot integrity*” (qdrant/lib/collection/src/operations/snapshot_ops.rs:77-80; qdrant/lib/storage/src/content_manager/snapshots/recover.rs:136-146). It is optional and comes from the same party as the file. **Rescoring:** PQ and SQ default to no rescoring:

```
// .../quantized/quantized_vectors/accessors.rs:16-22
QuantizedVectorStorage::ScalarRam(_) => false, ... PQRam(_) => false, ...
QuantizedVectorStorage::BinaryRam(_) => true,
```

The effective choice is params...rescore.unwrap_or(default_rescoring) (qdrant/lib/segment/src/index/vector_index_search_common.rs:64-72), and oversampling defaults to 1.0 (qdrant/lib/segment/src/types.rs:588-597). With PQ enabled, the compressed scores are the final scores unless the query asks for rescoring. A consumer who changes the collection’s quantization configuration forces a rebuild through the config-mismatch optimizer (qdrant/lib/shard/src/optimizers/config_mismatch_optimizer.rs:110-141).

Q5. PQ sampling uses a thread-local OS-seeded RNG: rand::seq::index::sample(&mut rand::rng(), count, sample_size) (qdrant/lib/quantization/src/encoded_vectors_pq.rs:365-366). Empty clusters are reseeded with rand::rng() (qdrant/lib/quantization/src/kmeans.rs:114-115), and the SQ quantile sample also uses rand::rng() (qdrant/lib/quantization/src/quantile.rs:290). No seed is stored in Metadata. **Qdrant codebooks cannot be reproduced by recomputation**, so a malicious codebook cannot be told apart from an unlucky honest one that way.

Q6. Searching qdrant/lib/api and qdrant/src for centroid and codebook returns nothing, so there is no public API. Centroids are plain JSON in quantized.meta.json, and editing a snapshot is trivial.

3.6 Weaviate

Q1. ProductQuantizer.Fit truncates the training data to trainingLimit and fits one KMeansEncoder per segment (weaviate/adapters/repos/db/vector/compressionhelpers/product_quantization.go:401-457). Encode maps each segment to its nearest centroid (.../product_quantization.go:460-466). The encoder uses random initialization (.../compressionhelpers/kmeans_encoder.go:48-65). SQ, RQ, and BQ live in the same package.

Q2. The default vector index is HNSW (weaviate/entities/vectorindex/config.go:25). Every compressor is off by default: DefaultPQEnabled = false (weaviate/entities/vectorindex/hnsw/pq_config.go:28), DefaultSQEnabled = false (.../sq_config.go:17), DefaultRQEnabled = false (.../rq_config.go:22), and DefaultBQEnabled = false (.../bq_config.go:17). A server-wide default comes from the environment and is empty unless set:

```
// weaviate/usecases/config/environment.go:633-638
defaultQuantization := ""
if v := os.Getenv("DEFAULT_QUANTIZATION"); v != "" { ... }
```

ParseDefaultQuantization maps "pq", "sq", "rq-1/4/8", and "bq" to enabled flags; "" and "none" leave compression off (weaviate/entities/vectorindex/hnsw/config.go:345-381). In-repo documentation shows that setting ALLOWED_COMPRESSION_TYPES to a single entry seeds DEFAULT_QUANTIZATION (example rq-8) (weaviate/docs/usage_limits.md:111,121,130-139). That is an operator choice, not a default. We did not audit Weaviate Cloud defaults or the external documentation site. **Scale trigger:** once PQ is enabled, the index queue calls Upgrade as soon as AlreadyIndexed() > shouldUpgradeAt (weaviate/adapters/repos/db/vector_index_queue.go:285-302). For PQ that threshold is pqConfig.TrainingLimit (weaviate/adapters/repos/db/vector/hnsw/index.go:976), with a default of 100,000 (.../pq_config.go:34).

Q3. Backups list the HNSW commit-log directory (weaviate/adapters/repos/db/vector/hnsw/backup.go:77-79). The commit log records the PQ codebooks: AddPQCompression writes the header and then encoder.ExposeDataForRestore() for every segment encoder (weaviate/adapters/repos/db/vector/hnsw/commitlog/logger.go:84-103). On restart, the encoders are rebuilt from these bytes through NewProductQuantizerWithEncoders (.../product_quantization.go:243-253) instead of being retrained. Full-precision vectors remain in the object store.

Q4. In weaviate/usecases/backup/*.go, the only matches for checksum, verify, and integrity are coverage and coordination checks (weaviate/usecases/backup/coordinator_dedupe.go:449-451). There is no content verification. **However, Weaviate rescores by default:**

```
// weaviate/adapters/repos/db/vector/hnsw/search.go:188-195
func (h *hnsw) shouldRescore() bool {
    if h.compressed.Load() { if (h.sqConfig.Enabled && h.sqConfig.RescoreLimit == 0) || ... { return false } }
    return h.compressed.Load() && !h.doNotRescore }
```

The rescore distance fetches the full-precision vector through TempVectorForIDWithViewThunk (.../search.go:822-852). doNotRescore defaults to the Go zero value (weaviate/adapters/repos/db/vector/hnsw/index.go:180). A malicious PQ codebook in Weaviate can therefore only change *which candidates reach the rescoring stage*, not their final order.

Q5. kmeans.New sets Seed: rand.Uint64() (weaviate/adapters/repos/db/vector/kmeans/kmeans.go:163), which feeds a PCG (.../kmeans.go:50). The training sample comes from a sparse Fisher-Yates sampler that uses global rand.Intn (weaviate/adapters/repos/db/vector/common/iterator.go:46; used in weaviate/adapters/repos/db/vector/hnsw/compress.go:48-75). PQData has no seed field (weaviate/entities/vectorindex/compression/pq_data.go:41-50). Training is not reproducible.

Q6. NewKMeansEncoderWithCenters(k, d, segment, centers) (.../compressionhelpers/kmeans_encoder.go:41-45) and NewProductQuantizerWithEncoders are internal Go APIs. weaviate/adapters/handlers/rest/operations has no REST operation that references centroids or codebooks. An attacker would have to craft commit logs inside a backup.

4 Threats to the attack premise

1. **PQ is opt-in in every repository.** No code path we traced turns on product quantization without the user naming it: a FAISS factory string, a Qdrant quantization_config, Weaviate pq.enabled or DEFAULT_QUANTIZATION=pq, or Milvus index_type=IVF_PQ/HNSW_PQ.
2. **The one default-on quantizer (Milvus) is SQ with refine.** It uses 4-bit uniform SQ plus an FP16 refine stage (milvus/pkg/util/paramtable/autoindex_param_nocuda.go:26). SQ has far fewer degrees of freedom than a PQ codebook: in effect, per-dimension ranges. This matches our prior finding that calibration ranges, not weights, are the malicious degree of freedom. Refine should re-rank candidates at FP16, which limits the attack to suppression. We could not verify the refine semantics because they live in nowhere, outside the repo.
3. **Default rescoring in Weaviate** with full-precision vectors (weaviate/adapters/repos/db/vector/hnsw/search.go:188-195,822-852) blocks promotion. A document that is not truly close cannot enter the final top-*k* once rescored, because only candidate recall can be manipulated.

4. **Scale and index gating.** Qdrant quantizes only indexed segments ($\geq 10,000$ KB, `qdrant/lib/shard/src/optimizers/config.rs:17`). Weaviate PQ trains only after 100,000 indexed vectors (`weaviate/entities/vectorindex/hnsw/pq_config.go:34`). Small RAG corpora may never reach the quantized path.
5. **Framework defaults are exact.** LangChain `InMemoryVectorStore` and LlamaIndex `SimpleVectorStore` are exact and full precision. The LlamaIndex Milvus integration even forces FLAT over Milvus's own SQ default (`.../milvus/base.py:1406`).
6. **Rebuild on configuration change (Qdrant).** A consumer who restores a snapshot and then sets their own quantization configuration triggers a rebuild that discards the shipped codebook (`qdrant/lib/shard/src/optimizers/config_mismatch_optimizer.rs:110-141`).
7. **Verification is possible in the vector DBs.** Qdrant snapshots, Weaviate's object store, and Milvus binlogs all keep the original vectors next to the compressed index. A consumer *could* re-encode them and compare. None of them does so, but the data needed for detection is present. This is not true of FAISS IVFPQ or LlamaIndex+FAISS.
8. **FAISS training is deterministic by default (seed=1234).** With the training set and parameters, an honest codebook can be recomputed exactly, and a malicious one would differ. The parameters are not serialized, however, and FAISS IVFPQ files contain no training vectors.
9. **The attacker who ships the artifact has more power than the attack needs.** In every Q3 path, the party shipping the file can edit codes or ID mappings directly (FAISS `add_sa_codes` and `update_entries`; Qdrant JSON metadata). The quantization-conditioned framing adds value only against a consumer who spot-checks exact behavior, or who checks that the codes are consistent with the shipped codebook. No repo performs either check today. This should be stated explicitly in the threat model.
10. **Unaudited code outside the repositories.** LangChain's FAISS wrapper (`langchain_community`), Milvus's quantizers (knowhere), pymilvus quick setup, and hosted-service defaults (Zilliz, Weaviate Cloud, Qdrant Cloud) are not in these repositories and were not audited.

5 Strongest instantiation

Repo: LlamaIndex + FAISS. **Index type:** `IndexIVFPQ`, optionally with OPQ, e.g. `faiss.index_factory(d, "OPQ16,IVF1024,PQ16")`. **Configuration:** the documented persist-and-reload pattern.

Why this one:

1. *The shipped artifact is the only copy of the vectors.* IVFPQ stores codes only (`faiss/faiss/impl/index_write.cpp:801-810`), and LlamaIndex removes embeddings from the docstore (`llama_index/core/indices/vector_store/base.py:196-199`). The consumer has nothing to verify against unless they re-embed the corpus.
2. *Loading does not depend on the index type.* `from_persist_path` accepts whatever `faiss.read_index` returns (`.../faiss/base.py:113-117`). A consumer who followed the `IndexFlatL2` example receives a PQ index silently.
3. *There is no rescoring.* IVFPQ search ranks by PQ-approximated distance unless the builder wrapped it in `IndexRefine`. Promotion and suppression both work.
4. *Codebooks and codes can be written directly* (`faiss/faiss/impl/ProductQuantizer.cpp:76-80`; `faiss/faiss/Index.h:430`). The attacker does not need to steer k-means.
5. *Training parameters are not recorded* in the file, so “re-train and compare” requires guessing the parameters.

Consumer steps that land on a PQ index they did not build (the pattern in `llama_index/docs/examples/vector_stores/FaissIndexDemo.ipynb:177-181`):

```
# 1. Obtain ./storage from a third party (teammate, data vendor, model/dataset hub, CI artifact).
# It contains default_vector_store.json (a FAISS binary written by faiss.write_index),
# docstore.json (nodes WITHOUT embeddings), index_store.json.
from llama_index.vector_stores.faiss import FaissVectorStore
```

```

from llama_index.core import StorageContext, load_index_from_storage
vector_store = FaissVectorStore.from_persist_dir("./storage") # faiss.read_index, no checks
storage_context = StorageContext.from_defaults(vector_store=vector_store, persist_dir="./storage")
index = load_index_from_storage(storage_context=storage_context)
# 2. Query. Retrieval runs on the attacker's PQ codebook/codes; the docstore maps
# positional FAISS ids -> node text. No step inspects index type, centroids, or codes.
retriever = index.as_retriever(similarity_top_k=5)

```

Runner-up instantiations. (i) *Qdrant with PQ enabled*: the consumer recovers a collection snapshot from a URL (qdrant/src/actix/api/snapshot_api.rs:251). `quantized.meta.json` is loaded as-is (`.../quantized_vectors/load.rs:32-35`), and PQ queries do not rescore by default (`.../accessors.rs:16-22`). This case is weaker than FAISS because the original vectors ship alongside the codes, so verification is possible, and because a configuration change forces a rebuild. (ii) *Milvus default*: any collection whose vector index was created without `index_type` gets HNSW_SQ/SQ4U, and `RestoreExternalSnapshot` copies prebuilt index files with only an ownership check. Quantization is on by default here, but it is SQ with FP16 refine, and the quantizer code is outside the audited repository.

Implication for the threat model. The honest framing is: *“quantized indexes are opt-in, or scale-triggered once opted in, everywhere except Milvus’s SQ default. Once a compressed index exists, it is a trusted, shippable artifact that no audited system verifies.”* The attack targets *configured* deployments at scale and pre-built index distribution. It does not target a default-on PQ supply chain.